

Final Project Short Narrative

Jack Underhill

November 30, 2025

WSU Campus Graph GPS

For my graph theory project, I decided to build a “WSU-GPS” style application for the Pullman campus. The idea is to overlay a graph structure on a campus map, treat campus locations as vertices and walking paths as weighted edges, and then use graph algorithms to suggest optimal walking routes. I wanted the project to be more than a static diagram, so I implemented it as an interactive web app that visualizes both the graph and the pathfinding process in real time.

The application is called *WSU Campus Graph GPS*. When a user opens the site, they see an image of the WSU Pullman campus with a graph drawn on top of it. The user can select two locations on campus, either by typing their names in the navigation bar (navbar) or by clicking vertices directly on the graph. Once the start and end vertices are chosen, the app runs Dijkstra’s algorithm on the underlying graph and highlights the shortest path. Along the way, the interface shows the algorithm’s “thinking”: which vertices are being visited, which edges are being considered, and the final distance between the two locations.

Overall design and technology stack

I implemented the project as a single-page web application using Vite + React for the frontend. I chose React because I am already comfortable building interactive UI components with it, and Vite gives fast build and development tooling. For styling, I relied on *Tailwind CSS* utility classes, which let me lay out the UI quickly and keep most of the styling co-located with the JSX.

The layout is simple on purpose. At the top there is a dark navbar that holds the “From”, “To”, and “Distance” elements. Below that, the rest of the screen is dedicated to the campus map. The map itself is just an image, but I treat it as a “world space” with a fixed width and height in internal coordinates. The graph (its vertices and edges) is defined in that world coordinate system. The React components then convert world coordinates into screen coordinates based on the current camera position and zoom level.

This separation between world space and screen space is important for two reasons. First, it lets me change the zoom and pan of the camera without having to modify the underlying graph data. Second, it keeps the logic consistent: a vertex at (x, y) always means the same point on the actual campus image, regardless of how the user is currently viewing the map.

Graph model and data structures

At the core of the project is a weighted graph $G = (V, E)$. Each *vertex* represents a campus location, such as a building or field. Every vertex stores:

- a unique `id` (used internally),
- a human-readable `name` (used in the UI and in the search box),
- a 2D position (x, y) in world coordinates, which corresponds to the map image.

Each *edge* represents a direct walking path between two locations. An edge stores:

- its own **id**,
- the endpoints u and v (the ids of the two vertices it connects),
- a numeric **weight** that represents the walking distance along that path.

The graph is stored in a dedicated module. Internally, I keep a dictionary of vertices keyed by id, a dictionary of edges keyed by id, and an adjacency list that maps each vertex id to a list of outgoing edges. The adjacency list is of the form

$$\text{adjacency}[u] \rightarrow [(v, w(u,v), \text{id}(e)), \dots]$$

where each triple $(v, w(u,v), \text{id}(e))$ corresponds to an edge $e = \{u, v\}$ with current vertex u , neighbor vertex v , edge weight $w(u,v)$, and the unique identifier of that edge.

This structure lets the pathfinding algorithm ask “who are my neighbors and what do the edges cost?” in constant or near-constant time. It also makes it easy to compute each vertex’s *degree* by looking at the length of its adjacency list. In the UI, every vertex renders a small badge next to it that shows its degree, so users can see at a glance which parts of the campus graph are more connected.

All edges in this project are treated as *undirected*. In practice, that means if I add an edge between vertices A and B , I insert an entry in the adjacency list for both directions. Conceptually, this matches walking paths on campus: if you can walk from one location to another, you can usually walk back the same way.

User interface and controls

The navbar is the main control surface for selecting routes. There are two text inputs labeled *From* and *To*. When the user starts typing into either box, a search routine runs over the set of vertices. It ranks suggestions by how well they match:

1. exact match on the id or name,
2. prefix match,
3. substring match.

This gives a fairly intuitive experience: if you know the name of a building, you can just type a few letters and pick it from the list. Once the user selects a suggestion, either by pressing Enter or clicking, it commits that vertex as the start or end of the route. As soon as both endpoints are set, the app triggers the pathfinding routine.

For users who prefer to interact directly with the map, there is an alternative interaction pattern. They can simply click on a vertex node. The first click sets the *start* vertex (displayed in green). Clicking a different vertex sets the *end* vertex (displayed in red) and automatically runs the pathfinding. If a full route already exists and the user clicks another vertex, the app treats that as a new start vertex and restarts the selection process. This gives a nice, fluid way to explore different routes without having to clear forms manually.

The map itself supports *mouse drag* and *touch drag* for panning, and *mouse wheel* or *pinch gestures* for zooming. To handle this, I wrote a custom hook that tracks a “camera” state: a center point in world coordinates and a scalar zoom factor. All the overlays (vertices, edges, and labels) are rendered in an SVG layer that uses the camera transform to map world coordinates to on-screen positions. I also clamp the camera so that panning cannot move the view completely off the map; this keeps the user from getting lost in an empty void.

Graph visualization and interaction

One of my goals for this project was not only to compute shortest paths, but also to make the underlying graph structure obvious and educational. To do that, I built several visualization features.

Every vertex is drawn as a small circle with a label. For the label, I show the vertex's *degree*. This is computed directly from the adjacency list, so it updates consistently with the graph data. Seeing the degree visually helps connect graph theory concepts to the physical layout of a campus (for example, major intersections naturally have a higher degree).

Edges are drawn as lines between the positions of their endpoint vertices. When the user hovers over an edge, I render a small pill-shaped label at the edge's midpoint showing its *weight*, which corresponds to the distance along that path. Hovering a vertex not only highlights the node itself but also highlights all incident edges. This makes it easy to see how local structure fits into the global network.

When a shortest path has been computed, the vertices and edges that lie along that route are highlighted with distinct colors. The start and end vertices are also styled differently (for example, green for the start and red for the destination), so it is clear where the route begins and ends.

Pathfinding with Dijkstra's algorithm

The central algorithmic piece is *Dijkstra's algorithm* for single-source shortest paths on a weighted graph with non-negative edge weights. Its time complexity is $O((V + E)\log V)$, for V vertices and E edges. Given a start vertex s and an end vertex t , the algorithm:

1. Initializes `dist[v] = Infinity` for all vertices v , except `dist[s] = 0`.
2. Uses a priority queue to repeatedly extract the unvisited vertex with the smallest tentative distance.
3. For each neighbor u of the current vertex v , it checks whether going through v gives a shorter path:

`if dist[v] + weight(v, u) < dist[u],`

then it updates `dist[u]` and records v as the predecessor of u .

4. Once all reachable vertices are processed, or once t has been finalized, it stops and reconstructs the shortest path by walking back through the predecessor pointers from t to s .

In my implementation, this logic lives in a dedicated hook that is responsible for running the algorithm, collecting the results, and exposing them to the rest of the app. The hook returns:

- `dist`: a map from vertex id to final distance,
- `pathVertices`: the ordered list of vertices on the shortest path,
- `pathEdges`: the ordered list of edges along that path,
- `visitedOrder` and a list of `steps`, which describe, in sequence, which vertices were visited and which edges were relaxed.

The `steps` array is used purely for visualization. Instead of immediately drawing the final path, the app first plays back the algorithm's reasoning. It iterates through the steps and temporarily highlights the "current" vertex and the edge being examined. This makes it clear which areas of the graph are explored first and how Dijkstra expands outward from the start node. Only after this traversal animation finishes does the app switch to showing the final clean path from start to end.

Because all of my edge weights are non-negative distances, Dijkstra's algorithm is guaranteed to find the true shortest route. In the context of this project, that means if a user picks two campus locations, the highlighted path represents the minimum total walking distance under the graph model I defined.

Challenges and implementation details

There were a few interesting implementation challenges in this project.

The first was *coordinate mapping*. The campus map is a static image, but I needed to be able to place vertices accurately on top of it. I solved this by defining a fixed world width and height that match the proportions of the image. I then manually chose (x, y) coordinates for each vertex in that world space. The camera hook takes those world coordinates, subtracts the camera center, applies the zoom transform, and then converts the result into pixel positions in the SVG overlay. Getting this right required some trial and error, especially to keep the graph aligned when the window resizes or when the user zooms in and out.

The second challenge was designing the *search and suggestion* behavior. I wanted typing “todd” or “todd hall” to quickly locate the right vertex. To do this, I wrote a small search function that scores each vertex based on how well its id or name matches the input string. Exact and prefix matches get higher scores than simple substring matches. The function then sorts the candidates and returns the best few as suggestions. This is not a full-blown search engine, but it is responsive and feels good to use.

Another challenge was handling *state transitions* cleanly. For example, when the user selects a new start vertex after already having a completed path, I needed to clear the existing route, reset any animations, and recompute the shortest path from the new starting point. I ended up centralizing this logic in my pathfinding hook and the map component. The hook exposes clear actions like “setStart”, “setEnd”, and “reset”, and the UI components just call those at the appropriate times.

Finally, I spent some time polishing the *hover and highlight* behavior to make the graph easier to read. I wrote a small hook to track which vertex or edge is currently hovered and used that to conditionally apply Tailwind classes to the SVG elements. This gives subtle effects like thickening an edge on hover and brightening the corresponding vertex, which help guide the user’s eye without overcomplicating the design.

Possible extensions and reflection

There are several directions I could extend this project in the future. One idea is to incorporate *elevation* or “Z-level” weights, so routes that involve steep hills or stairs could be penalized. Another is to add filters for *accessible routes*, for example excluding edges that represent staircases. It would also be possible to generalize the app so that other campuses or custom maps could be loaded by swapping out the underlying graph and base image.

Overall, this project met the goal of turning the abstract idea of a graph $G = (V, E)$ into something visual and practical. By combining a fairly standard shortest-path algorithm with an intuitive UI and simple animations, I was able to show how graph theory can support real-world problems like campus navigation. Building the project forced me to think carefully about data structures (like adjacency lists), algorithms (Dijkstra’s), and user interaction patterns (selection, zoom, and hover), and to integrate all of that into a cohesive tool. I am happy with where it landed, and I see it as a solid starting point for more advanced graph-based applications.

Live Links

You can view the project on my website: wsu-gps-graph-theory.netlify.app/.

You can view my video demo of the project at: youtu.be/B2fOKqGwaTA/.